

notificationapp por dentro

Un recorrido capa por capa: qué hace cada archivo, qué anotación resuelve qué problema, y por qué está armado así y no de otra forma.

FRAMEWORK

Spring Boot 4.1.1

LENGUAJE

Java 17 · Maven

PERSISTENCIA

**PostgreSQL 16 ·
JPA/Hibernate 7**

MENSAJERÍA

RabbitMQ (AMQP)

DOCS

**OpenAPI 3 /
Swagger UI**

TESTS

**JUnit 5 ·
Testcontainers**

Cómo leer esto. Las secciones siguen el camino de una petición HTTP: entra por el *controller*, baja al *service*, toca el *repository* y la base, y vuelve. Cada sección tiene tres partes: *qué es*, *cómo está hecho acá* (con el código real del repo) y *por qué*, casi siempre con un ejemplo de la vida real. Los recuadros verdes explican una decisión de diseño; los azules son ejemplos del mundo real; los rojos son la trampa que se evita.

LA REGLA DE ORO

Cada capa habla **solo con la de abajo** y a través de **tipos propios**. El controller no sabe SQL; el service no sabe HTTP; la entidad no sabe JSON. Cuando eso se respeta, cambiar una pieza (de REST a mensajería, de Postgres a otra base, de Basic a JWT) no obliga a tocar las demás.

Sección 01

El punto de entrada y el contenedor de Spring

Todo arranca en una clase con un `main` de tres líneas:

NotificationappApplication.java

```
@SpringBootApplication
public class NotificationappApplication {
    public static void main(String[] args) {
        SpringApplication.run(NotificationappApplication.class, args);
    }
}
```

`@SpringBootApplication` es tres anotaciones en una: `@ComponentScan` (buscá clases anotadas en este paquete y los de abajo), `@EnableAutoConfiguration` (mirá qué hay en el classpath y configurá lo obvio: si ves `postgresql` armá un `DataSource`, si ves `spring-web` levánta Tomcat en el 8080) y `@Configuration` (esta clase puede definir beans).

Bean, contenedor e inyección de dependencias

Un **bean** es un objeto que crea y administra Spring, no vos con `new`. Al arrancar, Spring construye un **contenedor** con todos los beans (los `@RestController`, `@Service`, `@Repository`, `@Component`, `@Configuration`) y resuelve quién necesita a quién. En este proyecto la inyección es siempre **por constructor**:

UserServiceImpl.java – constructor

```
private final UserRepository userRepository;
private final UserMapper userMapper;
private final PasswordEncoder passwordEncoder;

public UserServiceImpl(UserRepository userRepository, UserMapper userMapper,
                       PasswordEncoder passwordEncoder) {
    this.userRepository = userRepository;
    this.userMapper = userMapper;
    this.passwordEncoder = passwordEncoder;
}
```

POR QUÉ POR CONSTRUCTOR Y NO @AUTOWIRED EN EL CAMPO

Los campos son `final`: el objeto nace completo o no nace. Las dependencias quedan explícitas en la firma (si el constructor tiene 8 parámetros, la clase hace demasiado y se nota). Y en un test unitario construís la clase con `new` pasando dobles de prueba, sin levantar Spring.

MUNDO REAL

Hoy `NotificationService` guarda en base y listo. Mañana tiene que publicar un mensaje en RabbitMQ. Como el controller depende de la *interfaz* `NotificationService`, le inyectás otra implementación y el controller ni se entera. Eso es lo que compra la inyección de dependencias: poder cambiar el cableado sin abrir las clases que ya funcionan.

Sección 02

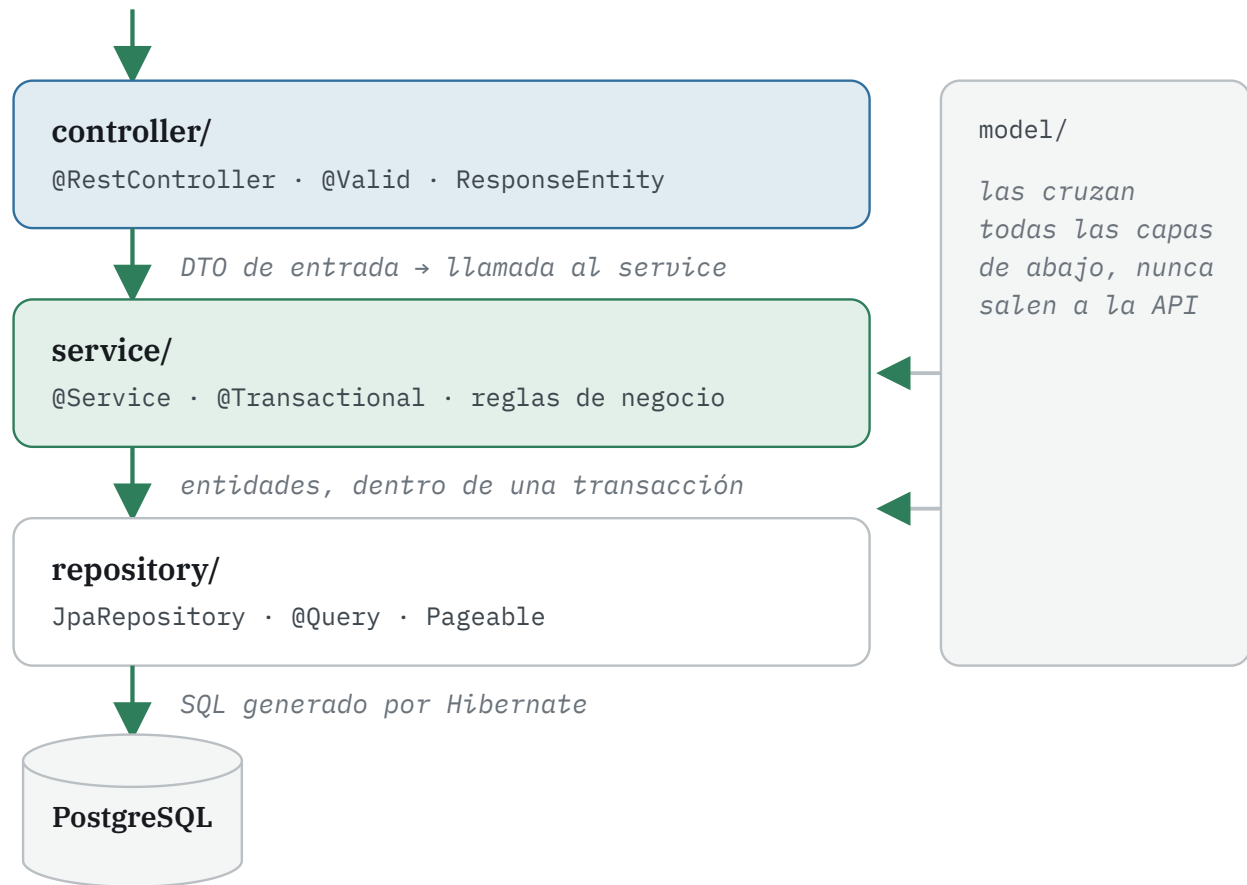
La arquitectura en capas

El paquete `com.alejandro.notificationapp` está partido por responsabilidad, no por entidad:

estructura de paquetes

controller/ → habla HTTP: recibe DTOs, devuelve DTOs y códigos de estado
service/ → la lógica de negocio y las transacciones
repository/ → acceso a datos (Spring Data JPA)
model/ → entidades JPA + enums (la capa "Model" del MVC)
dto/ → objetos de entrada/salida de la API (records)
mapper/ → traduce entidad ↔ DTO
exception/ → el manejo de errores centralizado
config/ → seguridad, OpenAPI, serialización de paginado

HTTP · JSON



la respuesta vuelve por el mismo camino, convertida a DTO en el service

Una petición baja capa por capa; cada frontera cambia de tipo de dato.

POR QUÉ PARTIR POR CAPA

Cada archivo tiene una sola razón para cambiar. Un bug de "devuelve 200 cuando debería 404" se arregla en el controller o el `GlobalExceptionHandler`, nunca en la query. Una query lenta se optimiza en el repository sin riesgo de romper una regla de negocio. El equipo puede tocar capas distintas sin pisarse.

Sección 03

Capa Model — las entidades JPA

Dos entidades: `User` (quién recibe) y `Notification` (qué se manda). JPA es el estándar de Java para mapear objetos a tablas; Hibernate es la implementación que trae Spring Boot.

User.java – campos y anotaciones (getters/setters omitidos)

```
@Entity
@Table(name = "users",
        uniqueConstraints = @UniqueConstraint(name = "uk_users_email", columnNames = "email"))
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(nullable = false, updatable = false)
    private UUID id;

    @Column(nullable = false, length = 320)
    private String email;

    @Column(nullable = false, length = 120)
    private String fullName;

    private String phoneNumber;    // canal SMS
    private String pushToken;      // canal PUSH

    /** Hash BCrypt, nunca la contraseña en claro. */
    @Column(nullable = false, length = 100)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Role role;             // USER | ADMIN

    @Column(nullable = false)
    private boolean enabled;

    @CreationTimestamp @Column(nullable = false, updatable = false)
    private Instant createdAt;

    @UpdateTimestamp @Column(nullable = false)
    private Instant updatedAt;

    protected User() { }          // requerido por JPA

    public User(String email, String fullName, String hashedPassword) {
        this.email = email;
        this.fullName = fullName;
        this.password = hashedPassword;
        this.role = Role.USER;
        this.enabled = true;
    }
}
```

Las decisiones, una por una

ID como UUID generado

`@GeneratedValue(strategy = GenerationType.UUID)`: la clave la genera Hibernate, no una secuencia de la base.

MUNDO REAL

Con IDs autoincrementales (1, 2, 3...) cualquiera que vea `/api/users/42` deduce que tenés ~42 usuarios y prueba `/api/users/43`. Con UUID no hay nada que adivinar. Además, si mañana hay que fusionar datos de dos entornos o generar el ID en el cliente antes de guardarlo, con autoincremental chocan las claves.

Enums guardados como texto

`@Enumerated(EnumType.STRING)` guarda "EMAIL", no 0.

LA TRAMPA QUE SE EVITA

El default de JPA es `ORDINAL`: guarda la *posición* del valor en el enum. El día que alguien agrega `WHATSAPP` entre `EMAIL` y `SMS`, todas las filas que decían "SMS" ahora significan "WHATSAPP". Silencioso e irreversible. Con `STRING` el orden del enum no importa.

La relación al destinatario

Notification.java

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "recipient_id", nullable = false)
private User recipient;
```

Muchas notificaciones apuntan a un usuario. En la tabla `notifications` hay una columna `recipient_id` con foreign key. `fetch = LAZY` significa "no traigas el usuario de la base hasta que alguien pida `notification.getRecipient()`".

POR QUÉ LAZY

Con `EAGER` (el default en `@ManyToOne`), cada vez que traés una notificación se hace un JOIN a `users` aunque no lo necesites. Multiplicá eso por una lista de 100 y son 100 JOINS regalados. `LAZY` te obliga a decidir explícitamente cuándo querés el usuario — y ese "cuándo" acá es el mapper, todavía dentro de la transacción.

Timestamps automáticos y tipo `Instant`

`@CreationTimestamp` y `@UpdateTimestamp` (de Hibernate) llenan `createdAt` / `updatedAt` solos. Se usa `Instant` (un punto en la línea de tiempo UTC), no `LocalDateTime`.

MUNDO REAL

`LocalDateTime` no tiene zona horaria: "2026-09-01 15:00" ¿de dónde? Si el server está en UTC y el dev en Buenos Aires, los logs y la base discrepan 3 horas y nadie sabe cuál es la verdad. `Instant` es siempre el mismo instante mundial; se convierte a la zona del usuario solo al mostrarlo.

Constructor `protected` vacío + constructor "de verdad"

JPA necesita un constructor sin argumentos para instanciar la entidad al leerla de la base (por eso está, pero `protected` para que el resto del código no lo use). El constructor público exige lo mínimo para que una `Notification` nazca en un estado válido: destinatario, asunto, mensaje y canal — y arranca en `PENDING`.

Lógica en la entidad, no "objeto anémico"

Notification.java

```
public void changeStatus(NotificationStatus newStatus) {
    this.status = newStatus;
    if (newStatus == NotificationStatus.SENT && this.sentAt == null) {
        this.sentAt = Instant.now();
    }
}
```

La regla "cuando pasa a `SENT` por primera vez, registrá `sentAt`" vive *dentro* de la entidad. El service llama `notification.changeStatus(SENT)` y no puede olvidarse de setear `sentAt`, porque no es su responsabilidad.

Capa Repository — Spring Data JPA

UserRepository.java

```
public interface UserRepository extends JpaRepository<User, UUID> {

    Optional<User> findByEmailIgnoreCase(String email);

    boolean existsByEmailIgnoreCase(String email);

}
```

Es una **interfaz**, sin implementación. Al arrancar, Spring Data genera en memoria una clase que la implementa. `JpaRepository<User, UUID>` ya trae `save`, `findById`, `findAll(Pageable)`, `delete`, etc. Los métodos que agregamos se traducen por su nombre: `existsByEmailIgnoreCase` → `SELECT count(*) > 0 FROM users WHERE lower(email) = lower(?)`. No se escribe SQL.

Cuando el nombre no alcanza: `@Query`

NotificationRepository.java

```
@Query("""
    select n from Notification n
    where (:recipientId is null or n.recipient.id = :recipientId)
        and (:status is null or n.status = :status)
        and (:channel is null or n.channel = :channel)
    """)
Page<Notification> search(@Param("recipientId") UUID recipientId,
                        @Param("status") NotificationStatus status,
                        @Param("channel") NotificationChannel channel,
                        Pageable pageable);
```

Es **JPQL** (consulta sobre entidades, no sobre tablas: `Notification`, no `notifications`). El truco `:param is null or campo = :param` hace que cada filtro sea opcional: si el controller no manda `status`, esa línea no filtra nada. Un solo método cubre "todas", "las de este usuario", "las fallidas de este usuario", etc.

Paginación: `Page` y `Pageable`

Ningún endpoint de listado devuelve una `List` pelada. Devuelven `Page<T>`: una porción (por defecto 20) más metadata (total, página actual, total de páginas).

MUNDO REAL

Un `findAll()` sobre una tabla de dos millones de notificaciones trae dos millones de objetos a memoria y probablemente tira el proceso con `OutOfMemoryError`. Con `Pageable`, el SQL lleva `LIMIT 20 OFFSET ...` y el response es chico y predecible. Es de esas cosas que hay que dejar bien desde el día uno porque después duele.

Sección 05

DTOs — por qué la API no expone las entidades

Un DTO (*Data Transfer Object*) es el objeto que viaja por la API. Están en `dto/`, son `record` (inmutables, sin getters a mano) y hay uno de entrada y otro de salida:

`dto/CreateUserRequest.java` · `dto/UserResponse.java`

```
public record CreateUserRequest(
    @NotBlank @Email @Size(max = 320) String email,
    @NotBlank @Size(max = 120) String fullName,
    @NotBlank @Size(min = 8, max = 72) String password,
    @Size(max = 30) String phoneNumber,
    @Size(max = 512) String pushToken
) {}

public record UserResponse(
    UUID id, String email, String fullName, String phoneNumber,
    String pushToken, Role role, boolean enabled,
    Instant createdAt, Instant updatedAt
) {} // ← no existe el campo password
```

LA TRAMPA QUE SE EVITA

Si el endpoint devolviera la entidad `User` directamente, el JSON incluiría `password` (el hash). Y el día que alguien agregue `passwordResetToken` o `failedLoginAttempts` a la entidad, esos campos aparecen en la respuesta sin que nadie lo haya decidido. El DTO de salida es una *lista blanca*: solo sale lo que está escrito ahí.

POR QUÉ SEPARAR ENTRADA DE SALIDA

Lo que el cliente *puede mandar* y lo que la API *devuelve* no son lo mismo. En el alta, el cliente manda `password` pero no `id` ni `createdAt` (los pone el server). En la respuesta es al revés. Dos records distintos evitan campos "que a veces sí y a veces no".

MUNDO REAL

Si la API expone la entidad, el contrato con el front queda pegado al esquema de la base. Renombrás una columna para una migración y rompés la app móvil que está en revisión de Apple hace una semana. Con DTOs, la base evoluciona por dentro y el contrato público cambia solo cuando vos querés.

Sección 06

Bean Validation — validar en el borde

Las anotaciones `@NotBlank`, `@Email`, `@Size`, `@NotNull` sobre los campos del record son **Jakarta Bean Validation**. Se disparan cuando el controller marca el parámetro con `@Valid`:

UserController.java

```
@PostMapping
public ResponseEntity<UserResponse> create(
    @Valid @RequestBody CreateUserRequest request, UriComponentsBuilder uriBuilder) {
    ...
}
```

Si el body no cumple, Spring lanza `MethodArgumentNotValidException` *antes* de entrar al método, y el `GlobalExceptionHandler` (sección 10) la convierte en un 400 con la lista de campos que fallaron.

POR QUÉ ACÁ Y NO EN LA BASE O EN EL SERVICE

La base también rechazaría un email de 500 caracteres, pero con un error de constraint ilegible y un 500. Validar en el borde da un mensaje claro y accionable (`"password: el tamaño debe estar entre 8 y 72"`) y le ahorra a las capas de abajo tener que desconfiar de sus entradas.

Mappers – la traducción entidad ↔ DTO

NotificationMapper.java

```
@Component
public class NotificationMapper {

    private final UserMapper userMapper;

    public NotificationMapper(UserMapper userMapper) { this.userMapper = userMapper; }

    public NotificationResponse toResponse(Notification n) {
        return new NotificationResponse(
            n.getId(),
            userMapper.toResponse(n.getRecipient()), // reusa el mapper de User
            n.getSubject(), n.getMessage(),
            n.getChannel(), n.getStatus(),
            n.getCreatedAt(), n.getUpdatedAt(), n.getSentAt());
    }
}
```

La conversión vive en un solo lugar. Si mañana `NotificationResponse` tiene que ocultar el `message` para notificaciones de otro usuario, se cambia acá y vale para todos los endpoints. `NotificationMapper` compone `UserMapper` en vez de repetir la conversión del destinatario.

POR QUÉ UNA CLASE Y NO MAPSTRUCT

Con dos entidades, un mapper a mano son 15 líneas legibles y sin magia. MapStruct (que genera el mapper en tiempo de compilación) empieza a valer la pena con 15 entidades y jerarquías anidadas. Meterlo ahora es complejidad sin retorno.

Capa Service – negocio y transacciones

Cada service es una **interfaz** (`NotificationService`) más una **implementación** (`NotificationServiceImpl`). El controller depende de la interfaz.

NotificationServiceImpl.java

```
@Service
@Transactional(readOnly = true)
public class NotificationServiceImpl implements NotificationService {

    @Override
    @Transactional // sobrescribe: esta sí escribe
    public NotificationResponse create(CreateNotificationRequest request) {
        User recipient = userRepository.findById(request.recipientId())
            .orElseThrow(() -> ResourceNotFoundException.of(
                "Usuario destinatario", request.recipientId()));
        Notification notification = new Notification(
            recipient, request.subject(), request.message(), request.channel());
        return notificationMapper.toResponse(
            notificationRepository.saveAndFlush(notification));
    }

    @Override
    @Transactional
    public NotificationResponse updateStatus(UUID id, NotificationStatus status) {
        Notification notification = findOrThrow(id);
        notification.changeStatus(status);
        return notificationMapper.toResponse(
            notificationRepository.saveAndFlush(notification));
    }

    private Notification findOrThrow(UUID id) {
        return notificationRepository.findById(id)
            .orElseThrow(() -> ResourceNotFoundException.of("Notificación", id));
    }
}
```

Qué es una transacción

`@Transactional` envuelve el método en una transacción de base: o pasa todo, o no pasa nada. Si `create` encuentra al usuario, construye la notificación y falla al guardar, no queda nada a medias.

MUNDO REAL

El ejemplo clásico es la transferencia bancaria: restar de una cuenta y sumar a otra tienen que ser atómicos. Acá el caso es más sutil pero igual de real: leer el usuario, validar reglas y escribir la notificación deben ver una foto consistente de la base y confirmarse juntos.

POR QUÉ `READONLY = TRUE` EN LA CLASE

La mayoría de los métodos solo leen. `readOnly = true` le avisa a Hibernate que no haga *dirty checking* (no compare cada objeto por si cambió) y al driver que puede optimizar. Los métodos que escriben (`create`, `update`, `delete`) ponen `@Transactional` a secas y ganan.

Por qué `saveAndFlush` y no `save`

`save()` solo *agenda* el INSERT; Hibernate recién genera `createdAt` / `updatedAt` al hacer *flush*, que por defecto ocurre al cerrar la transacción. Como el mapeo a DTO pasa *antes* de ese cierre, los timestamps salían en `null` en la respuesta del alta. `saveAndFlush()` fuerza el flush dentro del método: el objeto ya tiene los valores cuando lo mapeamos.

El patrón `findOrThrow`

`findById` devuelve `Optional<Notification>`. En vez de repetir el `.orElseThrow(...)` en cada método, hay un helper privado. Lanza `ResourceNotFoundException`, que la capa de errores convierte en 404. El service **no sabe** que existe el número 404 — solo dice "esto no está".

Capa Controller — hablar HTTP bien

NotificationController.java

```
@RestController
@RequestMapping("/api/notifications")
public class NotificationController {

    @PostMapping
    public ResponseEntity<NotificationResponse> create(
        @Valid @RequestBody CreateNotificationRequest request,
        UriComponentsBuilder uriBuilder) {
        NotificationResponse created = notificationService.create(request);
        URI location = uriBuilder.path("/api/notifications/{id}")
            .buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);    // 201 + Location
    }

    @GetMapping("/{id}")
    public NotificationResponse getById(@PathVariable UUID id) {
        return notificationService.getById(id);    // 200
    }

    @GetMapping
    public Page<NotificationResponse> list(
        @RequestParam(required = false) UUID recipientId,
        @RequestParam(required = false) NotificationStatus status,
        @RequestParam(required = false) NotificationChannel channel,
        @PageableDefault(size = 20, sort = "createdAt",
            direction = Sort.Direction.DESC) Pageable pageable) {
        return notificationService.list(recipientId, status, channel, pageable);
    }

    @PatchMapping("/{id}/status")
    public NotificationResponse updateStatus(
        @PathVariable UUID id,
        @Valid @RequestBody UpdateNotificationStatusRequest request) {
        return notificationService.updateStatus(id, request.status());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable UUID id) {
        notificationService.delete(id);
        return ResponseEntity.noContent().build();    // 204
    }
}
```

El controller **no tiene lógica**: recibe, delega en el service, elige el código HTTP y devuelve. Cada verbo tiene una semántica que conviene respetar:

Verbo	Uso acá	Idempotente	Éxito
POST	crear	No – dos POST = dos recursos	201 Created + Location
GET	leer	Sí – no cambia nada	200 OK
PUT	reemplazar el user completo	Sí – repetirlo deja el mismo estado	200 OK
PATCH	cambiar solo el estado	Sí (en este caso)	200 OK
DELETE	borrar	Sí – borrar dos veces = borrado	204 No Content

MUNDO REAL – POR QUÉ 201 Y EL HEADER LOCATION

Un cliente hace `POST /api/notifications`. Con `201` sabe que se creó (no solo que "salió bien"), y el header `Location: /api/notifications/b25e...` le dice exactamente dónde quedó, sin tener que parsear el body ni adivinar la URL. Un webhook o un script de integración se apoyan en eso.

MUNDO REAL – POR QUÉ IMPORTA LA IDEMPOTENCIA

La app móvil manda `DELETE /api/notifications/123`, el celular pierde señal, no llega la respuesta, y reintenta. Con `DELETE` idempotente el segundo intento devuelve 204 igual y no pasa nada raro. Si "borrar" estuviera en un `POST` que además incrementa un contador, el reintento lo rompe.

El listado usa `@PageableDefault`: `?page=0&size=20&sort=createdAt,desc` sale gratis, y los filtros son `@RequestParam(required = false)` – opcionales, como la query del repository.

Sección 10

Manejo de errores centralizado

Ni un `try/catch` en los controllers. Una sola clase con `@RestControllerAdvice` intercepta las excepciones de todas las capas y las convierte en el mismo formato de error:

exception/ApiError.java – el cuerpo de error uniforme

```
public record ApiError(
    Instant timestamp, int status, String error,
    String message, String path, List<FieldViolation> fieldErrors
) {
    public record FieldViolation(String field, String message) {}
}
```

exception/GlobalExceptionHandler.java (extracto)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(...) { /* 404 */ }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ApiError> handleDuplicate(...) { /* 409 */ }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleBodyValidation(...) { /* 400 + campos */ }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ApiError> handleDataIntegrity(...) { /* 409 */ }

    @ExceptionHandler(Exception.class) // red de contención
    public ResponseEntity<ApiError> handleUnexpected(Exception ex, ...) {
        log.error("Error no controlado en {}", request.getRequestURI(), ex);
        return build(HttpStatus.INTERNAL_SERVER_ERROR, "Ocurrió un error inesperado", ...);
    }
}
```

Excepción	HTTP	Cuándo
<code>ResourceNotFoundException</code>	404	el <code>id</code> no existe
<code>DuplicateResourceException</code>	409	email ya registrado
<code>MethodArgumentNotValidException</code>	400	body inválido (con lista de campos)
<code>MethodArgumentTypeMismatchException</code>	400	<code>/api/users/abc</code> – "abc" no es UUID
<code>HttpMessageNotReadableException</code>	400	JSON roto o enum inexistente (<code>"channel":"CARTA"</code>)
<code>DataIntegrityViolationException</code>	409	borrar un user con notificaciones (FK)
<code>Exception</code> (catch-all)	500	cualquier otra – se logea completa, se devuelve genérica

LA TRAMPA QUE SE EVITA

El handler de `Exception` logea el stacktrace *del lado del server* pero al cliente le manda `"Ocurrió un error inesperado"`. Nunca un stacktrace en el response: filtra nombres de clases, versiones, rutas de archivos y a veces fragmentos de query – un mapa para un atacante.

POR QUÉ UN FORMATO ÚNICO

El front escribe *un* parser de errores: mira `fieldErrors` para pintar los campos del formulario en rojo, y `message` para el toast. Si cada endpoint devolviera errores con forma distinta, el manejo de errores del cliente sería un árbol de `if` infinito.

Sección 11

Seguridad – Spring Security

Spring Security se mete como una **cadena de filtros** delante de los controllers: cada request pasa por autenticación y autorización antes de llegar al código. La config vive en `config/SecurityConfig.java`:

SecurityConfig.java

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/actuator/health", "/actuator/info").permitAll()
            .requestMatchers(DOCS_WHITELIST).permitAll() // swagger
            .requestMatchers(HttpMethod.POST, "/api/users").permitAll() // registro
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}

@Bean
PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

BCrypt para las contraseñas

El service nunca guarda la contraseña en claro: `passwordEncoder.encode(raw)` produce un hash BCrypt (con *salt* incluido y un costo configurable que lo hace deliberadamente lento).

MUNDO REAL

Se filtra la base — pasa todo el tiempo. Con SHA-256 pelado, un atacante prueba millones de contraseñas por segundo contra el dump y saca las fáciles en minutos. BCrypt con costo 10 hace ~10 hashes por segundo por core: el mismo ataque pasa de minutos a años. El *salt* por usuario impide además las *rainbow tables*.

Stateless: sin sesión en el server

`SessionCreationPolicy.STATELESS`: el server no guarda una `HttpSession`. Cada request se autentica sola (hoy con HTTP Basic, mañana con un JWT en el header `Authorization`).

MUNDO REAL

En producción corren 4 instancias detrás de un balanceador. Con sesión en memoria, el usuario que se logueó en la instancia 2 explota si la request siguiente cae en la 3 (o necesitás *sticky sessions* y un Redis compartido). Sin estado, cualquier instancia atiende cualquier request y escalar es agregar réplicas.

CSRF deshabilitado — y por qué acá está bien

EL RAZONAMIENTO

CSRF protege contra que *el navegador* adjunte credenciales ambientales (una cookie de sesión) en una request forjada desde otro sitio. Una API stateless que se autentica con un token en el header `Authorization` no tiene ese vector: otro origen no puede leer ni setear ese header. Si esta API usara cookies de sesión + browser, CSRF tendría que quedar **activo**.

Hoy HTTP Basic, mañana JWT

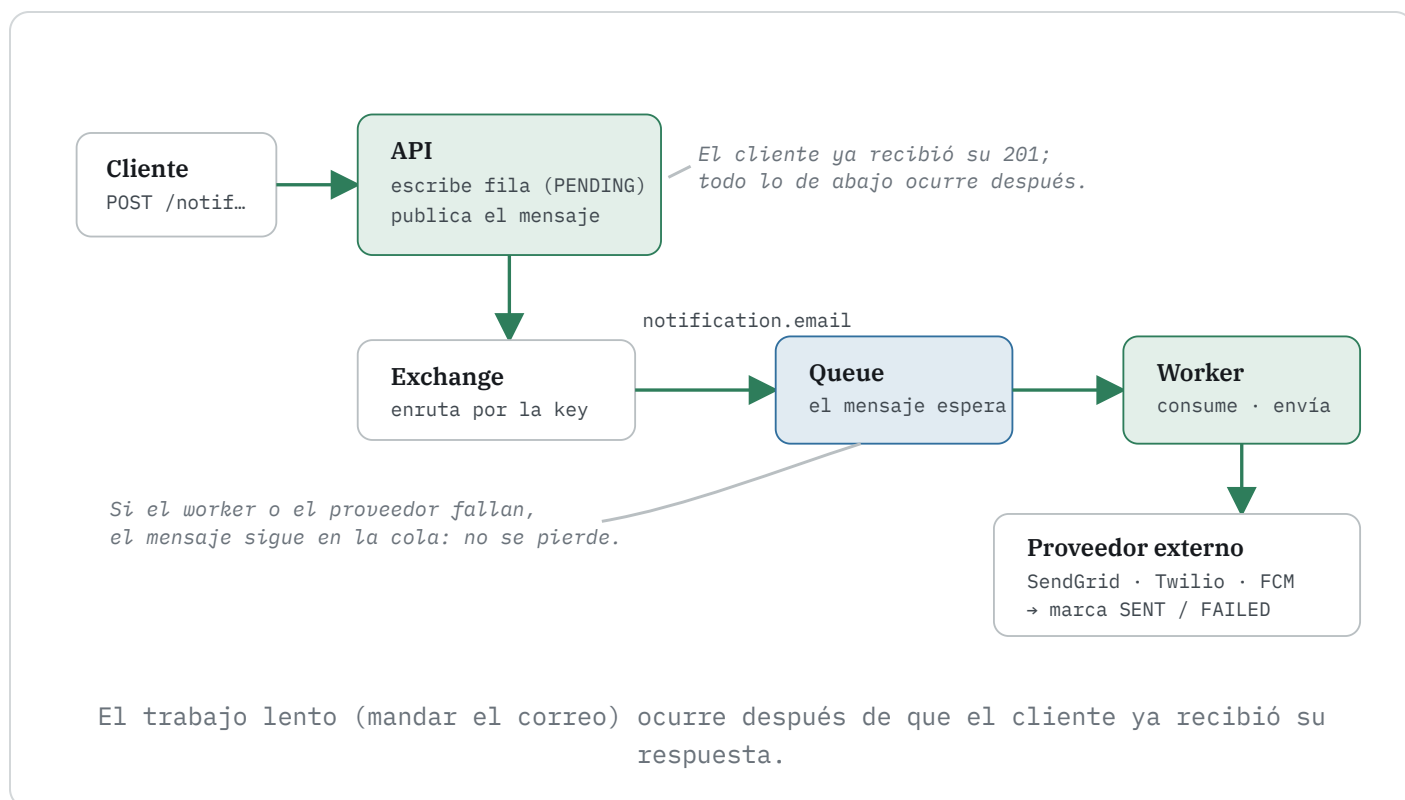
Por ahora hay un único usuario administrador en memoria (`UserDetailsService` con `app.admin.*`) y se entra con HTTP Basic (`admin / admin`). El `.env.example` ya reserva `JWT_SECRET` y `JWT_EXPIRATION_MS`: el próximo paso es un `UserDetailsService` que lea de `UserRepository` (`findByEmailIgnoreCase` ya está) y un filtro que valide el token JWT — se enchufa en esta misma cadena, antes de `UsernamePasswordAuthenticationFilter`.

Sección 12

RabbitMQ — mensajería asíncrona

RabbitMQ es un **message broker**: un intermediario donde un proceso deja un mensaje y otro lo levanta cuando puede. El vocabulario:

Término	Qué es
Producer	quien publica el mensaje (acá: la API, al crear la notificación)
Exchange	la "central de correo": recibe el mensaje y decide a qué cola(s) va
Routing key	la etiqueta del mensaje (p. ej. <code>notification.email</code>) que el exchange usa para enrutar
Queue	la cola donde el mensaje espera hasta ser consumido
Consumer / worker	el proceso que saca mensajes de la cola y hace el trabajo (llamar a SendGrid, a Twilio...)
Ack	el consumer confirma "procesado"; sin ack, RabbitMQ re-entrega el mensaje



Por qué asíncrono y no mandar el email en el mismo request

MUNDO REAL – EL PICO DE BLACK FRIDAY

A las 00:00 se disparan 80.000 notificaciones "tu pedido fue confirmado". Si cada `POST` esperara a que SendGrid responda (200–800 ms cada uno), los threads de Tomcat se agotan, la API deja de responder *todo* — incluido el login — y el pico se transforma en caída. Con la cola, la API solo escribe una fila y publica un mensaje (~milisegundos); los workers drenan la cola a su ritmo. La cola **amortigua** el pico.

MUNDO REAL – EL PROVEEDOR SE CAE

Twilio tiene un incidente de 15 minutos. Sin cola, esos 15 minutos de SMS se pierden (o cada request devuelve error y el usuario lo ve). Con cola, los mensajes se acumulan; cuando Twilio vuelve, el worker los procesa. Sumás una *dead-letter queue* para los que fallan N veces y tenés reintentos y una bandeja de "revisar a mano" sin escribir un scheduler.

POR QUÉ DESACOPLAR

El código que atiende el `POST` no necesita saber *cómo* se manda un email. Publica "hay que notificar a este usuario por EMAIL" y sigue. El worker de email, el de SMS y el de push evolucionan, se despliegan y escalan por separado. Incluso podés tener varios workers compitiendo por la misma cola para ir más rápido.

Cómo encaja en este código hoy

La dependencia `spring-boot-starter-amqp` ya está en el `pom.xml` y Testcontainers ya levanta un RabbitMQ para los tests. El modelo lo anticipa: el estado `PENDING → SENT / FAILED` de `Notification` es exactamente el ciclo de vida de un mensaje en la cola. Falta la pieza del medio: al crear la notificación, publicar el mensaje (`RabbitTemplate.convertAndSend(...)`); y un `@RabbitListener` que consuma, llame al proveedor y haga `notification.changeStatus(SENT)`.

Sección 13

Actuator — la app se cuenta a sí misma

Spring Boot Actuator expone endpoints operativos bajo `/actuator`. En este proyecto, solo dos:

```
application.properties
```

```
management.endpoints.web.exposure.include=health,info
# RabbitMQ todavia no se usa: no marcar el health global como DOWN.
management.health.rabbit.enabled=false
```

- `GET /actuator/health` → `{"status":"UP"}` si la app y sus dependencias (la base) responden; `DOWN` + 503 si no.
- `GET /actuator/info` → metadata de build (versión, commit) si se configura.

MUNDO REAL – KUBERNETES

K8s hace dos chequeos periódicos: *liveness* ("¿está vivo? si no, reiniciá el pod") y *readiness* ("¿puede recibir tráfico? si no, sacalo del balanceador"). Ambos pegan a `/actuator/health`. Si la conexión a Postgres se cae, `health` devuelve 503, K8s deja de mandarle tráfico a ese pod, y cuando la base vuelve el pod se reincorpora solo. Sin este endpoint, K8s manda usuarios a una instancia rota.

MUNDO REAL – MONITOREO

Agregando `micrometer-registry-prometheus` aparece `/actuator/prometheus` con métricas (requests por segundo, latencia p99, uso del pool de conexiones, memoria de la JVM). Prometheus las *scrapea* cada 15 s y Grafana las grafica. Es la diferencia entre "un usuario dice que anda lento" y "la latencia p99 saltó a las 14:32, justo con el deploy".

LA TRAMPA QUE SE EVITA

`exposure.include` es una lista blanca a propósito. Actuator también puede exponer `/actuator/env` (todas las variables de entorno, incluidos secretos), `/actuator/heapdump` (un volcado de memoria descargable) y `/actuator/loggers` (cambiar niveles de log en caliente). Exponer `*` a internet es regalar la llave.

POR QUÉ `MANAGEMENT.HEALTH.RABBIT.ENABLED=FALSE`

El starter de AMQP trae un health indicator para RabbitMQ. Como todavía no hay un broker en desarrollo, ese indicador daría `DOWN` y arrastraría el `health` global a `DOWN` — y entonces un K8s real no dejaría arrancar la app. Se apaga hasta que RabbitMQ sea de verdad una dependencia.

Testcontainers — tests con infra real

```
src/test/.../TestcontainersConfiguration.java
```

```
@TestConfiguration(proxyBeanMethods = false)
class TestcontainersConfiguration {

    @Bean @ServiceConnection
    RabbitMQContainer rabbitContainer() {
        return new RabbitMQContainer(DockerImageName.parse("rabbitmq:latest"));
    }

    @Bean @ServiceConnection
    PostgreSQLContainer postgresContainer() {
        return new PostgreSQLContainer(DockerImageName.parse("postgres:16-alpine"));
    }
}
```

Al correr los tests, Testcontainers arranca un PostgreSQL y un RabbitMQ reales en Docker, efímeros, y `@ServiceConnection` inyecta sus URLs/credenciales en el contexto de Spring automáticamente. Al terminar, los contenedores se destruyen.

LA TRAMPA QUE SE EVITA — "EN MI MÁQUINA CON H2 PASABA"

La tentación es testear contra H2 (una base en memoria). Pero H2 no es PostgreSQL: los tipos, las funciones (`ILIKE` , `JSONB` , `ON CONFLICT`), el comportamiento de las secuencias y hasta los mensajes de error de constraint son distintos. Un test que pasa con H2 y una feature que explota en producción es el clásico "pero los tests estaban verdes".

POR QUÉ TAMBIÉN SIRVE PARA DESARROLLAR

`./mvnw spring-boot:test-run` levanta la app *usando* esta config: un dev clona el repo, tiene Docker, y corre la app completa con su base y su broker sin instalar ni configurar nada. El `docker-compose.yml` queda para quien prefiera una base persistente entre reinicios.

OpenAPI / Swagger — el contrato vivo

config/OpenApiConfig.java

```
@Bean
OpenAPI notificationappOpenAPI() {
    return new OpenAPI()
        .info(new Info().title("Notification App API").version("v1")
            .description("CRUD de usuarios y notificaciones."))
        .components(new Components().addSecuritySchemes("basicAuth",
            new SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("basic")))
        .addSecurityItem(new SecurityRequirement().addList("basicAuth"));
}
```

`springdoc-openapi` escanea los controllers, los DTOs y sus anotaciones de validación en tiempo de arranque y arma el documento OpenAPI 3 en `/v3/api-docs`. Swagger UI (en `/swagger-ui.html`) lo pinta como una consola donde probar cada endpoint. El `SecurityScheme` hace que el botón *Authorize* pida usuario/contraseña y los mande en cada llamada.

POR QUÉ GENERADO Y NO ESCRITO A MANO

Un README con ejemplos de `curl` se desactualiza en la primera semana. Este documento sale del código: si agregás un campo al DTO o un endpoint, la doc cambia sola. El front puede incluso generar su cliente HTTP desde el `/v3/api-docs`.

Sección 16

Configuración y entornos

application.properties

```
spring.datasource.url=${DATABASE_URL:jdbc:postgresql://localhost:5432/notificaciones_db}
spring.datasource.username=${DATABASE_USERNAME:notificaciones_user}
spring.datasource.password=${DATABASE_PASSWORD:cambiame123}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false

app.admin.username=${APP_ADMIN_USER:admin}
app.admin.password=${APP_ADMIN_PASSWORD:admin}
```

`${VAR:default}`

Toma la variable de entorno `VAR`; si no está, usa el default. En local funciona sin configurar nada; en producción el operador setea `DATABASE_URL`, `DATABASE_PASSWORD`, etc. El `.env.example` documenta qué variables existen; el `.env` real (con los secretos) está en `.gitignore`.

MUNDO REAL

Una contraseña de base commiteada al repo queda *para siempre* en el historial de git, aunque después la borres. Los secretos van por variable de entorno (o Vault, o los secrets de K8s), nunca en un archivo versionado.

`ddl-auto=update`

Hibernate ajusta el esquema al arrancar comparándolo con las entidades. Cómodo en desarrollo.

LA TRAMPA QUE SE EVITA

`update` nunca borra ni cambia una columna existente, y no es reproducible. En producción se usa **Flyway** o **Liquibase**: migraciones versionadas (`V1__crear_users.sql`, `V2__agregar_indice.sql`) que se aplican en orden, quedan en git y corren igual en cada entorno. Está anotado como pendiente.

`open-in-view=false`

POR QUÉ APAGARLO

Con *Open Session In View* (el default de Spring Boot), la sesión de Hibernate queda abierta hasta que termina de renderizarse la respuesta, así que un `getRecipient()` lazy en el mapper "funciona" — disparando queries sueltas fuera de toda transacción, el terreno del problema N+1. Apagándolo, si el código accede a algo lazy fuera de la transacción, *falla en los tests* y te obliga a cargar por adelantado lo que la respuesta necesita. Por eso el mapeo a DTO se hace dentro del método `@Transactional` del service.

Sección 17

El `pom.xml`

El `<parent>` es `spring-boot-starter-parent`: un BOM (*Bill of Materials*) que fija versiones compatibles de cientos de librerías, así no hay que elegir las a mano. Las dependencias son **starters** — paquetes temáticos:

Starter	Qué trae	Estado
<code>data-jpa</code>	JPA, Hibernate, pool de conexiones HikariCP	en uso
<code>web</code> (vía websocket)	Spring MVC, Tomcat embebido, Jackson	en uso
<code>security</code>	la cadena de filtros, BCrypt	en uso
<code>validation</code>	Jakarta Bean Validation (Hibernate Validator)	en uso
<code>actuator</code>	<code>/health</code> , <code>/info</code> , métricas	en uso
<code>amqp</code>	cliente RabbitMQ, <code>RabbitTemplate</code> , <code>@RabbitListener</code>	listo, sin cablear
<code>mail</code>	<code>JavaMailSender</code>	listo, para el worker de email
<code>websocket</code>	STOMP/SockJS	listo, para push in-app
<code>springdoc-openapi</code>	OpenAPI 3 + Swagger UI (v3.1.0, la línea que soporta Boot 4)	en uso

Detalles que valen: `postgresql` tiene `<scope>runtime</scope>` (el código compila contra JDBC, no contra el driver); `lombok` y `devtools` son `<optional>true</optional>` (no se propagan a quien use este artefacto, y devtools se desactiva solo al correr desde el jar); los `testcontainers-*` son `<scope>test</scope>`. Lombok está disponible pero las entidades usan getters explícitos a propósito — son parte del contrato de dominio y conviene verlos.

Sección 18

Cómo correr y probar

terminal

```
# App completa con Postgres + RabbitMQ efímeros (requiere Docker)
./mvnw spring-boot:test-run

# App contra infra externa (docker compose up -d primero)
./mvnw spring-boot:run

# Tests de integración
./mvnw test

# Empaquetar el jar ejecutable
./mvnw clean package
java -jar target/notificationapp-0.0.1-SNAPSHOT.jar
```

Con la app arriba:

- **Swagger UI:** `http://localhost:8080/swagger-ui.html` — botón *Authorize* con `admin` / `admin`.
- **OpenAPI JSON:** `http://localhost:8080/v3/api-docs`
- **Health:** `http://localhost:8080/actuator/health`

Flujo mínimo de humo: `POST /api/users` (público) → tomar el `id` → `POST /api/notifications` con ese `recipientId` → `PATCH /api/notifications/{id}/status` con `{"status": "SENT"}` → ver que `sentAt` quedó seteado.

POST /api/users

201 · sin auth

**POST
/api/notifications**

201 · Basic

**GET
/api/notifications**

200 · Page

PATCH .../status

200 · SENT

DELETE ...

204

Qué falta — roadmap

Pendiente	Dónde engancha
JWT (login + filtro de token)	<code>SecurityConfig</code> + <code>UserDetailsService</code> sobre <code>UserRepository</code>
Publicar a RabbitMQ al crear la notificación	<code>NotificationServiceImpl.create</code> → <code>RabbitTemplate</code>
Workers de email / SMS / push	nueva capa <code>messaging/</code> con <code>@RabbitListener</code> + <code>JavaMailSender</code>
Flyway en vez de <code>ddl-auto=update</code>	<code>src/main/resources/db/migration</code>
Tests por capa (<code>@WebMvcTest</code> , <code>@DataJpaTest</code> , unitarios de service)	<code>src/test</code>
Observabilidad: Micrometer + Prometheus, tracing	<code>pom.xml</code> + <code>management.*</code>

Glosario

Bean	objeto creado y administrado por el contenedor de Spring
IoC / DI	inversión de control / inyección de dependencias: las clases reciben sus colaboradores en vez de crearlos
ORM	mapeo objeto-relacional: objetos Java ↔ filas de tablas (JPA es el estándar, Hibernate la implementación)
JPQL	lenguaje de consultas de JPA, escrito sobre entidades y no sobre tablas
DTO	objeto que viaja por la API; desacopla el contrato público del modelo interno
Idempotente	ejecutar la operación una o N veces deja el mismo estado
Transacción	unidad atómica contra la base: confirma todo o revierte todo
Broker	intermediario de mensajería (RabbitMQ); desacopla productor y consumidor en el tiempo
Exchange / Queue	el exchange enruta; la queue almacena hasta que un consumer procesa
Dead-letter queue	cola a donde van los mensajes que fallaron demasiadas veces
Hash / salt	huella irreversible de la contraseña; el salt (aleatorio por usuario) impide precomputar ataques
BOM	<i>Bill of Materials</i> : POM que fija versiones compatibles de un set de librerías
Starter	dependencia "paraguas" de Spring Boot que trae todo lo necesario para un tema
Dialect	la variante de SQL que Hibernate genera para un motor concreto (Postgres ≠ H2 ≠ MySQL)
OSIV	<i>Open Session In View</i> : mantiene la sesión de Hibernate abierta durante el render; acá está apagado

notificationapp · guía del código · generada a partir del estado del repositorio. Todos los fragmentos son código real del proyecto; los nombres de clase y ruta son navegables en el IDE.